OpenTelemetry in Python

A Practical Tutorial Series

Part 1 — Introduction to Observability & OpenTelemetry

Series Overview

Part 1 — Introduction to Observability & OpenTelemetry ← You are here
Part 2 — Distributed Tracing with Python
Part 3 — Metrics Collection & Prometheus
Part 4 — Structured Logging & Correlation
Part 5 — The OpenTelemetry Collector
Part 6 — End-to-End Observability in Practice

Part 1 — Introduction to Observability & OpenTelemetry

What You Will Learn

- What observability is and why it matters for modern software
- The three pillars of observability: Traces, Metrics, and Logs
- What OpenTelemetry is, its history, and the problems it solves
- Core concepts: signals, instrumentation, exporters, and the Collector
- How to set up a Python project with the OpenTelemetry SDK
- Your first instrumented Python application

Prerequisites

This series is written for developers who are comfortable with Python and have a basic understanding of how web applications work. You do not need prior experience with observability tools, distributed tracing, or monitoring systems — we will build that knowledge from scratch.

> 🐍 **What you need installed**
> Python 3.8 or higher, pip, and a terminal. Docker is optional but recommended for running backends like Jaeger in later parts.

1.1 — The Problem: Flying Blind in Production

Imagine your web application starts responding slowly at 2 AM. Users are complaining. You SSH into your server and see high CPU usage — but why? Which service is responsible? Which database query is slow? Is it one user causing the load, or all of them?

Without the right tooling, answering these questions means digging through flat log files, guessing, and hoping you can reproduce the issue. This is what it means to fly blind in production.

Modern software is increasingly **distributed** — a single user request may touch several services before returning a response. Here is a typical e-commerce checkout flow:

```
User Browser
    |
    v
API Gateway          <- receives the HTTP request
    |
```

```
    v
Auth Service         <- validates the session token
    |
    v
Payment Service      <- charges the card
    |
    v
Database             <- records the transaction

Total response time: 4.2s   <- but which step caused the slowdown?
```

Traditional monitoring ("is the server up?") tells you *something is wrong*, but not *why* or *where*. That is the gap observability is designed to fill.

1.2 — What is Observability?

Observability is the ability to understand the internal state of a system by examining its external outputs. The term originally comes from control theory, but in software engineering it has a practical meaning: a system is observable if engineers can answer questions about its behavior without deploying new code.

A highly observable system lets you ask and answer questions like:

- Why did this request take 4 seconds instead of 200ms?
- Which service introduced the latency?
- Did the error rate increase after the last deployment?
- How many database connections are open right now?
- Is this bug affecting one user or everyone?

> 💡 **Monitoring vs Observability**
>
> Monitoring tells you when something is wrong based on pre-defined thresholds. Observability lets you explore and understand why something is wrong, even for problems you didn't anticipate. You need both — observability doesn't replace monitoring, it enhances it.

1.3 — The Three Pillars of Observability

Observability data typically falls into three categories, often called the three pillars. OpenTelemetry is built around all three. Before diving into each, here is a simple mental hook to keep them straight:

Logs	Metrics	Traces
What happened?	*How much / how often?*	*Where did the request go?*

Traces

A **trace** records the journey of a single request as it travels through your system. Each unit of work within that journey is called a **span**. Spans can be nested — a parent span might represent an HTTP request, while child spans represent a database query or an outgoing API call made during that request.

Traces answer: "What happened during this specific request, and how long did each step take?"

Span	Service	Duration	Status
`GET /checkout`	api-gateway	312ms	OK
`→ validate_cart()`	cart-service	45ms	OK
`→ → SELECT * FROM cart`	postgres	38ms	OK
`→ charge_payment()`	payment-service	240ms	OK
`→ send_email()`	email-service	27ms	OK

Metrics

A **metric** is a numerical measurement recorded over time. Metrics are aggregated — rather than storing every individual event, they capture summaries like counts, averages, or percentiles.

Metrics answer: "How is my system performing right now, and how has that changed over time?"

Common metric types include:

- **Counter** — a value that only goes up, e.g. total requests served
- **Gauge** — a value that goes up and down, e.g. active database connections
- **Histogram** — distribution of values, e.g. request latency percentiles (p50, p95, p99)

Logs

A **log** is a timestamped record of a discrete event. Logs are the oldest form of observability data — and still essential. The key evolution in modern logging is **structured logs**: instead of free-form text strings, structured logs are machine-readable records (typically JSON) with consistent fields.

Logs answer: "What exactly happened at this moment in time?"

🔗 **The power of correlation**

The three pillars become exponentially more useful when they are correlated. OpenTelemetry makes this easy: a trace ID can be embedded in every log line generated during that trace, and metrics can

be annotated with the same service attributes. When something goes wrong, you can jump from a metric spike → to the traces causing it → to the exact log lines within those traces.

1.4 — What is OpenTelemetry?

OpenTelemetry (often abbreviated OTel) is an open-source observability framework — a collection of APIs, SDKs, and tools for generating, collecting, and exporting telemetry data (traces, metrics, and logs) from your applications.

> **Important: OpenTelemetry does NOT store or display your data**
>
> This is the most common beginner misconception. OpenTelemetry is a producer of telemetry data — it generates and sends it. You still need a separate backend to store and visualize it: Jaeger for traces, Prometheus for metrics, or a commercial platform like Datadog or Honeycomb. Think of OTel as the pipeline, not the destination.

It is a Cloud Native Computing Foundation (CNCF) project and has become the industry standard for instrumentation. Major cloud providers (AWS, Google Cloud, Azure), APM vendors (Datadog, New Relic, Honeycomb, Grafana), and open-source tools all support it.

A Brief History

Before OpenTelemetry, every vendor had its own proprietary instrumentation library. Switching from Jaeger to Zipkin meant re-instrumenting your entire codebase. Two open-source projects — OpenTracing and OpenCensus — tried to fix this but had overlapping goals. In 2019, they merged to form OpenTelemetry, and it quickly became the standard.

The Core Promise: Vendor Neutrality

OpenTelemetry separates instrumentation from the destination of your data. You instrument your code once using the OTel API. Then, using configuration (not code changes), you choose where to send that data — Jaeger, Prometheus, Datadog, a self-hosted Collector, or all of the above simultaneously.

This means your instrumentation code is never locked to a vendor. Switching backends is a configuration change, not a rewrite.

Without OpenTelemetry	With OpenTelemetry
Instrument once per vendor	Instrument once, export anywhere
Switching vendors = rewrite	Switching vendors = config change
Fragmented APIs per signal type	Unified API for traces, metrics, logs

Vendor agents installed on every host	One Collector handles all signals
Lock-in to proprietary formats	Open standards (OTLP protocol)

1.5 — Key Concepts

Before writing any code, it helps to understand the main building blocks of OpenTelemetry. Here is how they all connect:

```
Your Application
     |
     |  OTel API calls (create spans, record metrics, emit logs)
     v
OTel SDK
     |
     |  processes, samples, batches telemetry data
     v
Exporter
     |
     |  serialises data into OTLP format
     v
OTel Collector  (optional but recommended)
     |
     |  receives, transforms, and fans-out
     v
Backend(s): Jaeger / Prometheus / Datadog / Grafana / ...
```

One concept at a time — here is what each layer does:

The API vs. the SDK

The **OTel API** is a thin, stable interface for instrumentation. It defines how you create spans, record metrics, and emit log records. Library authors instrument their code against the API.

The **OTel SDK** is the concrete implementation. It provides the actual machinery: samplers, processors, and exporters. Application developers configure and initialize the SDK at startup.

This separation means a library can be instrumented without pulling in a heavy dependency — the API package is intentionally lightweight.

Instrumentation

Instrumentation is the act of adding telemetry code to your application. OTel supports two approaches:

- **Auto-instrumentation** — a zero-code-change approach where OTel automatically instruments popular libraries (Flask, Django, SQLAlchemy, requests, etc.) using monkey-patching at startup.
- **Manual instrumentation** — you explicitly create spans and record metrics in your own code for custom business logic.

Most production applications use both: auto-instrumentation for the scaffolding, and manual spans for the parts that matter most to your domain.

Exporters

An **exporter** is a plugin that sends your telemetry data to a backend. OTel ships with exporters for many destinations: Jaeger, Zipkin, Prometheus, the OTel Collector (via OTLP), and a console exporter for development.

The Collector

The **OpenTelemetry Collector** is an optional but powerful component — a standalone service that receives telemetry from your applications, processes it, and forwards it to one or more backends. It decouples your application code from specific backends and enables features like batching, filtering, and fan-out to multiple destinations. We will cover it in depth in Part 5.

OTLP — The OpenTelemetry Protocol

OTLP (OpenTelemetry Protocol) is the standard wire format for sending telemetry data between OTel components. It works over gRPC or HTTP/JSON. When you configure an application to export to the Collector, it speaks OTLP. This protocol is now supported natively by most major APM vendors.

1.6 — Setting Up Your Python Project

Let's get your environment ready. We will create a small Python project and install the core OpenTelemetry packages.

Project Structure

```
otel-tutorial/
├── part1/
│   ├── app.py           # Our first instrumented application
│   └── requirements.txt
├── part2/               # (added in later parts)
└── docker-compose.yml   # (added in Part 2 for Jaeger)
```

Installing Dependencies

Create the project directory and install the required packages:

```
mkdir -p otel-tutorial/part1 && cd otel-tutorial/part1

# Create requirements.txt
cat > requirements.txt << EOF
```

```
opentelemetry-api==1.24.0
opentelemetry-sdk==1.24.0
opentelemetry-exporter-otlp==1.24.0
EOF

pip install -r requirements.txt
```

📦 **Package Overview**

opentelemetry-api provides the instrumentation interfaces. opentelemetry-sdk provides the runtime implementation. opentelemetry-exporter-otlp provides the OTLP exporter for sending data to the Collector or a compatible backend. We will add more packages (Flask instrumentation, Prometheus exporter, etc.) in later parts.

1.7 — Your First Instrumented Application

Let's write a simple Python script that manually creates a trace with a few spans and outputs them to the console. This gives you a clear, dependency-free view of what telemetry data actually looks like.

app.py

```
# part1/app.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import (
    BatchSpanProcessor,
    ConsoleSpanExporter,
)

# 1. Create a TracerProvider — the entry point for tracing.
#    In production you would configure exporters here.
provider = TracerProvider()

# 2. Add a ConsoleSpanExporter so we can see output in the terminal.
provider.add_span_processor(
    BatchSpanProcessor(ConsoleSpanExporter())
)

# 3. Register the provider globally.
trace.set_tracer_provider(provider)

# 4. Get a Tracer — your tool for creating spans.
#    The name should identify your component or service.
tracer = trace.get_tracer("my-first-service")


def process_order(order_id: str):
    # 5. Start a span. The 'with' block automatically ends it.
    with tracer.start_as_current_span("process_order") as span:
        # 6. Set attributes — key/value metadata on the span.
        span.set_attribute("order.id", order_id)
        span.set_attribute("order.source", "web")

        # Simulate some nested work
        validate_order(order_id)
```

```
        charge_payment(order_id)


def validate_order(order_id: str):
    # Child span — automatically parented to process_order
    with tracer.start_as_current_span("validate_order") as span:
        span.set_attribute("order.id", order_id)
        # Simulate work
        import time; time.sleep(0.02)


def charge_payment(order_id: str):
    with tracer.start_as_current_span("charge_payment") as span:
        span.set_attribute("order.id", order_id)
        span.set_attribute("payment.method", "credit_card")
        import time; time.sleep(0.05)


if __name__ == "__main__":
    process_order("ORD-1234")
    # Flush all pending spans before exit
    provider.shutdown()
```

Running the Application

```
python app.py
```

You will see JSON output in your terminal for each span — something like this (abbreviated):


```
{
    "name": "charge_payment",
    "context": {
        "trace_id": "0x4bf92f3577b34da6a3ce929d0e0e4736",
        "span_id": "0x00f067aa0ba902b7"
    },
    "parent_id": "0xa3ce929d0e0e4736",
    "start_time": "2024-03-01T10:00:00.123456Z",
    "end_time": "2024-03-01T10:00:00.173456Z",
    "attributes": {
        "order.id": "ORD-1234",
        "payment.method": "credit_card"
    },
    "status": {"status_code": "UNSET"}
}
```


Notice the key fields:

- `trace_id` — shared across all spans in the same request
- `span_id` — unique to this span
- `parent_id` — links this span to its parent, forming the trace tree
- `attributes` — the key/value metadata you set
- `start_time` and `end_time` — used to compute duration

1.8 — Adding a Span Event

Spans can also carry **events** — timestamped annotations that mark something of interest happening within a span, without creating a new span. They are useful for recording notable moments like a retry, a cache miss, or a warning.

```
def charge_payment(order_id: str):
    with tracer.start_as_current_span("charge_payment") as span:
        span.set_attribute("order.id", order_id)

        # Simulate a retry
        span.add_event(
            "payment.retry",
            attributes={"retry.attempt": 1, "retry.reason": "timeout"}
        )

        import time; time.sleep(0.05)
```

1.9 — Marking a Span as an Error

When something goes wrong, you should record the error on the span so it appears correctly in tracing UIs:

```
from opentelemetry.trace import StatusCode

def charge_payment(order_id: str):
    with tracer.start_as_current_span("charge_payment") as span:
        try:
            span.set_attribute("order.id", order_id)
            # ... payment logic ...
            raise ValueError("Card declined")
        except Exception as e:
            span.record_exception(e)   # Records stack trace as an event
            span.set_status(StatusCode.ERROR, str(e))
            raise
```

1.10 — Understanding trace_id and span_id

Before moving to Part 2, it is important to understand two fields that appear in every span — these are the glue that holds distributed tracing together.

trace_id — The Request Fingerprint

A `trace_id` is a unique identifier assigned to an entire request journey — from the moment it enters your system to the moment it completes. Every single span created during that journey shares the same trace_id, regardless of which service created it.

This is what makes distributed tracing powerful: you can collect all spans with the same trace_id from across dozens of services and reconstruct the full picture of what happened.

```
# One user clicks Checkout. This single trace_id ties everything together:

trace_id = 4bf92f3577b34da6a3ce929d0e0e4736

  api-gateway     span_id=00f067  parent=none      <- root span
  auth-service    span_id=a1b2c3  parent=00f067    <- child span
  payment-service span_id=d4e5f6  parent=00f067    <- child span
  postgres        span_id=789abc  parent=d4e5f6    <- grandchild span
```

span_id — The Individual Operation

A **`span_id`** uniquely identifies one specific span within a trace. Each span also records its **`parent_id`** — the span_id of whatever called it. This parent-child relationship is what lets tracing tools reconstruct the call tree shown in the diagram above.

A span with no parent_id is called the root span — typically the entry point of the request (e.g. the incoming HTTP request to your API gateway).

> **Why this matters for Part 2**
>
> In Part 2 we will see how trace_id and span_id are propagated across HTTP boundaries via request headers (W3C TraceContext format). When service A calls service B, it passes the current trace_id and span_id in a header. Service B reads this header and creates its spans as children, keeping the entire trace stitched together even across process boundaries.

Summary

In this first part you have learned:

- What observability is and how it differs from monitoring
- The three pillars — Traces, Metrics, and Logs — and what questions each answers
- What OpenTelemetry is, why it was created, and its core value proposition: vendor-neutral instrumentation
- Key OTel concepts: the API/SDK split, instrumentation types, exporters, and the Collector
- How to set up a Python project and write your first traced application with manual spans

> ⏭️ **What's Next: Part 2 — Distributed Tracing**
>
> In Part 2 we will instrument a real Flask web application with both auto- and manual instrumentation, explore context propagation across service boundaries, and visualize our traces in Jaeger running in Docker.

Further Reading

- OpenTelemetry official documentation: opentelemetry.io/docs
- OTel Python SDK on GitHub: github.com/open-telemetry/opentelemetry-python

- CNCF OpenTelemetry project page